LAPORAN PRAKTIKUM MACHINE LEARNING
CLASSIFICATION LOGISTIC REGRESSION

Oleh:

Nama : Dyelvio Wijaya

NIM : 211344012

Tanggal Praktikum : 24 Oktober 2023

Tanggal Pengumpulan Laporan : 27 Oktober 2023

PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG

OKTOBER 2023

A. Logistic Regression

Import Library

```
import pandas as pd
import pylab as pl
import numpy as np
import scipy.optimize as opt
from sklearn import preprocessing
%matplotlib inline
import matplotlib.pyplot as plt
```

Dowload the Telco Churn Data

```
!wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%203/data/ChurnData.csv"
```

Reading Data

```
churn_df = pd.read_csv("ChurnData.csv")
churn_df.head()
```

Output Program:

	tenure	age	address	income	ed	employ	equip	callcard	wireless	longmon	...	pager	internet	callwait	confer	ebill	loglong	logtoll	lninc	custcat	churn
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	4.40	...	1.0	0.0	1.0	1.0	0.0	1.482	3.033	4.913	4.0	1.0
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	9.45	...	0.0	0.0	0.0	0.0	0.0	2.246	3.240	3.497	1.0	1.0
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	6.30	...	0.0	0.0	0.0	1.0	0.0	1.841	3.240	3.401	3.0	0.0
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	6.05	...	1.0	1.0	1.0	1.0	1.0	1.800	3.807	4.331	4.0	0.0
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	7.10	...	0.0	0.0	1.0	1.0	0.0	1.960	3.091	4.382	3.0	0.0

Data pre-Processing dan Seleksi

```
churn_df = churn_df[['tenure', 'age', 'address', 'income', 'ed', 'employ', 'equip',   'callcard', 'wireless','churn']]
churn_df['churn'] = churn_df['churn'].astype('int') #Mengubah semua data dalam churn menjadi tipe integer
churn_df.head()
```

Output Program:

	tenure	age	address	income	ed	employ	equip	callcard	wireless	churn
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	1
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	1
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	0
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	0
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	0

B. Practice

How many rows and columns are in this dataset in total? What are the names of columns?

```
[12] # write your code here
     churn_df.shape #"untuk mengetahui jumlah baris dan kolom dalam DataFrame"
```

```
(200, 10)
```

Let's define X, and y for our dataset:

```
[ ] X = np.asarray(churn_df[['tenure', 'age', 'address', 'income', 'ed', 'employ', 'equip']])
    X[0:5] #"menghasilkan data dari indeks 0 hingga 4"
```

```
array([[ 11.,  33.,   7., 136.,   5.,   5.,   0.],
       [ 33.,  33.,  12.,  33.,   2.,   0.,   0.],
       [ 23.,  30.,   9.,  30.,   1.,   2.,   0.],
       [ 38.,  35.,   5.,  76.,   2.,  10.,   1.],
       [  7.,  35.,  14.,  80.,   2.,  15.,   0.]])
```

```
y = np.asarray(churn_df['churn'])
y [0:5] #"menghasilkan data dari indeks 0 hingga 4"
```

```
array([1, 1, 0, 0, 0])
```

Also, we normalize the dataset:

```
[16] from sklearn import preprocessing
     #"preprocessing.StandardScaler() membuat objek yang akan digunakan untuk melakukan penskalaan standar"
     #"fit() untuk menghitung rata-rata dan deviasi standar dari setiap kolom (fitur) dalam data 'X'"""
     #"transform() untuk menerapkan penskalaan standar pada data 'X' berdasarkan rata-rata dan deviasi standar"
     X = preprocessing.StandardScaler().fit(X).transform(X)
     X[0:5]
```

```
array([[-1.13518441, -0.62595491, -0.4588971 ,  0.4751423 ,  1.6961288 ,
        -0.58477841, -0.85972695],
       [-0.11604313, -0.62595491,  0.03454064, -0.32886061, -0.6433592 ,
        -1.14437497, -0.85972695],
       [-0.57928917, -0.85594447, -0.261522  , -0.35227817, -1.42318853,
        -0.92053635, -0.85972695],
       [ 0.11557989, -0.47262854, -0.65627219,  0.00679109, -0.6433592 ,
        -0.02518185,  1.16316   ],
       [-1.32048283, -0.47262854,  0.23191574,  0.03801451, -0.6433592 ,
         0.53441472, -0.85972695]])
```

C. Train/Test Dataset

We split our dataset into train and test set:

```
from sklearn.model_selection import train_test_split
#"train_test_split yang digunakan untuk membagi dataset menjadi dataset pelatihan dan dataset pengujian"
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=4)
print ('Train set:', X_train.shape,  y_train.shape)
print ('Test set:', X_test.shape,  y_test.shape)
```

```
Train set: (160, 7) (160,)
Test set: (40, 7) (40,)
```

Modeling

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
#C=0.01: Parameter ini mengontrol kekuatan regularisasi dalam model Regresi Logistik
#solver='liblinear': Ini adalah algoritma yang digunakan oleh model untuk menyelesaikan permasalahan optimisasi
#fit() dari model Regresi Logistik untuk melatih model
LR = LogisticRegression(C=0.01, solver='liblinear').fit(X_train,y_train)
LR
```

```
LogisticRegression(C=0.01, solver='liblinear')
```

```
yhat = LR.predict(X_test)
yhat
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0])
```

```
yhat_prob = LR.predict_proba(X_test)#metode yang digunakan untuk memprediksi probabilitas kelas
yhat_prob
```

```
array([[0.54132919, 0.45867081],
       [0.60593357, 0.39406643],
       [0.56277713, 0.43722287],
       [0.63432489, 0.36567511],
       [0.56431839, 0.43568161],
       [0.55386646, 0.44613354],
       [0.52237207, 0.47762793],
       [0.60514349, 0.39485651],
       [0.41069572, 0.58930428],
       [0.6333873 , 0.3666127 ],
       [0.58068791, 0.41931209],
       [0.62768628, 0.37231372],
       [0.47559883, 0.52440117],
       [0.4267593 , 0.5732407 ],
       [0.66172417, 0.33827583],
       [0.55092315, 0.44907685],
       [0.51749946, 0.48250054],
       [0.485743  , 0.514257  ],
       [0.49011451, 0.50988549],
       [0.52423349, 0.47576651],
       [0.61619519, 0.38380481],
       [0.52696302, 0.47303698],
       [0.63957168, 0.36042832],
       [0.52205164, 0.47794836],
       [0.50572852, 0.49427148],
       [0.70706202, 0.29293798],
       [0.55266286, 0.44733714],
       [0.52271594, 0.47728406],
       [0.51638863, 0.48361137],
       [0.71331391, 0.28668609],
       [0.67862111, 0.32137889],
       [0.50896403, 0.49103597],
       [0.42348082, 0.57651918],
       [0.71495838, 0.28504162],
       [0.59711064, 0.40288936],
       [0.63808839, 0.36191161],
       [0.39957895, 0.60042105],
       [0.52127638, 0.47872362],
       [0.65975464, 0.34024536],
       [0.5114172 , 0.4885828 ]])
```

D. Evaluasi

Jaccard Index

```
from sklearn.metrics import jaccard_score
jaccard_score(y_test, yhat,pos_label=0) #untuk menghitung skor Jaccard
```

```
0.7058823529411765
```

Confusion Matrix

```
from sklearn.metrics import classification_report, confusion_matrix
import itertools
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')

    print(cm)

    #menampilkan gambar atau citra dalam bentuk visual
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    #menampilkan judul
    plt.title(title)
    #menambahkan batang warna (colorbar) ke dalam plot atau subplot
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    #np.arange = membuat larik (array) berdasarkan jarak tertentu antara elemen-elemennya
    #mengatur atau menyesuaikan tanda-tanda (tick marks)
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)
```

```
    fmt = '.2f' if normalize else 'd'
    #.2f'. Ini berarti data akan dicetak dalam format pecahan dengan dua desimal
    #'d'. Ini berarti data akan dicetak sebagai bilangan bulat (integer)
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        #menempatkan teks di lokasi tertentu dalam gambar
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

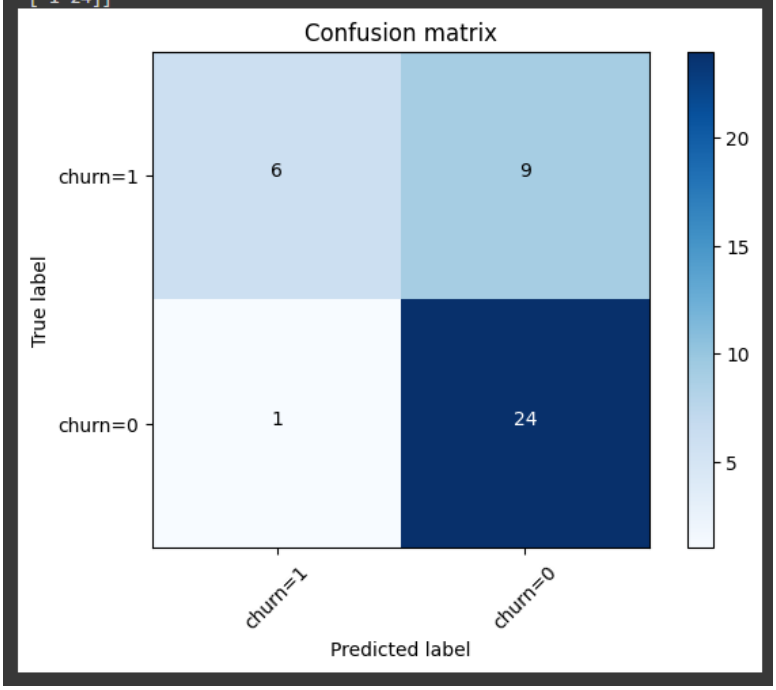
    plt.tight_layout()#untuk mengelola tata letak dalam matplotlib secara otomatis
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
#mencetak matriks konfusi (confusion matrix) berdasarkan hasil prediksi model
#dan nilai sebenarnya pada dataset pengujian
print(confusion_matrix(y_test, yhat, labels=[1,0]))
```

```
[[ 6  9]
 [ 1 24]]
```

```
# Compute confusion matrix
cnf_matrix = confusion_matrix(y_test, yhat, labels=[1,0])
#mengatur pengaturan cetak (printing options) untuk cara bilangan pecahan (floating-point numbers) ditampilkan dalam array
#precision=2 mengatur bahwa hanya dua digit di belakang tanda desimal yang akan ditampilkan ketika mencetak bilangan pecahan
np.set_printoptions(precision=2)


# Plot non-normalized confusion matrix
plt.figure()
#untuk membuat visualisasi matriks konfusi (confusion matrix) dalam pemodelan klasifikasi
plot_confusion_matrix(cnf_matrix, classes=['churn=1','churn=0'],normalize= False,  title='Confusion matrix')
```

```
Confusion matrix, without normalization
[[ 6  9]
 [ 1 24]]
```




```
print (classification_report(y_test, yhat))
#menghasilkan laporan klasifikasi yang memberikan berbagai metrik evaluasi klasifikasi
```

	precision	recall	f1-score	support
0	0.73	0.96	0.83	25
1	0.86	0.40	0.55	15
accuracy			0.75	40
macro avg	0.79	0.68	0.69	40
weighted avg	0.78	0.75	0.72	40

Log Loss

```
from sklearn.metrics import log_loss
log_loss(y_test, yhat_prob)
#menghitung log loss atau logaritma dari kerugian dalam masalah klasifikasi
```

```
0.6017092478101185
```

E. Practice

Try to build Logistic Regression model again for the same dataset, but this time, use different _solver_ and _regularization_values? What is new _logLoss_ value?

```
LR2 = LogisticRegression(C=0.01, solver='sag').fit(X_train,y_train)
yhat_prob2 = LR2.predict_proba(X_test)
#menampilkan nilai log_loss dengan dua angka dibelakang koma
print ("LogLoss: : %.2f" % log_loss(y_test, yhat_prob2))
```

```
LogLoss: : 0.61
```

F. Analisa

Dalam percobaan kali ini dalam salah satu jenis metode pembelajaran machine learning yang bertipe supervised yaitu Logistic Regression. Logistic Regression adalah sebuah metode yang digunakan untuk mengklasifikasi, yang dimana tujuannya yaitu untuk memprediksi kelas atau label kategori dari suatu data berdasarkan fitur-fitur atau variabel input yang ada. Dalam Logistic Regression ada beberapa library yang dibutuhkan yaitu diantaranya terdapat pandas, pylab, numpy, scipy.optimize, preprocessing dari sklearn, dan matplotlib.pyplot. Kemudian sama seperti Linear Regresi, perlu adanya sebuah dataset yang dapat digunakan untuk train dan juga testnya, maka dalam percobaan kali ini saya menggunakan sebuah dataset dari IBM bernama ChurnData.csv. Sama seperti linear regresi juga, untuk logistic regresi ini pun perlu disiapkan sebuah objek atau variabel yang akan menampung semua data dari data set, dalam percobaan kali digunakan churn_df dan untuk proses memasukkan dapat dilakukan dengan proses inisialisasi dan untuk proses pembacaan dataset dapat menggunakan fungsi yang ada di lib pandas yaitu read_csv.

Lalu untuk mengetahui isi data yang ada dalam dataset dengan cara memanggil sebagian saja data dapat menggunakan fungsi head(), ataupun dapat juga menggunakan proses pemanggilan dengan cara array, namun perlu adanya proses konversi dari tipe data yang ada dalam dataset tersebut menjadi sebuah dataset yang bertipe array. Dalam pemanggilan array dapat menggunakan cara seperti berikut X[Indeks Awal : Indeks Akhir], indeks awal adalah indeks ke berapa yang akan ditampilkan dan indeks akhir adalah indeks terakhir ke berapa yang akan dimunculkan. Kemudian jika ingin mengetahui jumlah baris dan kolom pada dataset dapat menggunakan fungsi shape.

Kemudian untuk menormalisasi dataset dapat melakukan tiga proses yaitu StandardScaler yaitu menghitung rata-rata dan deviasi standar dari setiap fitur dalam data pelatihan, dan kemudian menggunakannya untuk menyesuaikan (transform) data pelatihan dan data pengujian ke dalam distribusi yang telah dinormalisasi. Kemudian fit() untuk menghitung rata-rata dan deviasi standar dari setiap kolom (fitur) dalam data. Dan yang terakhir yaitu transform() untuk menerapkan penskalaan standar pada data berdasarkan rata-rata dan deviasi standar yang telah dihitung. Sehingga nantinya data akan memiliki rata rata 0 dan deviasi standar 1.

Lalu untuk membagi sebuah dataset untuk train dan test dapat menggunakan sebuah fungsi yaitu train_test_split. Kemudian untuk membuat sebuah model logistic regression dalam

percobaan kali ini digunakan sebuah solver liblinear dan juga sag dalam practice. Liblinear adalah salah satu solver yang tersedia di scikit-learn yang cocok untuk masalah regresi logistik dengan data yang tidak terlalu besar. Lalu untuk sag adalah salah satu solver yang tersedia di scikit-learn yang cocok untuk masalah regresi logistik dengan data yang besar.

Untuk mengetahui model yang kita buat apakah sudah baik atau tidak dapat menggunakan jaccard index yaitu skor Jaccard yang mengukur sejauh mana prediksi model cocok dengan nilai sebenarnya pada kelas yang ditentukan oleh pos_label. Skor Jaccard dihitung sebagai perbandingan jumlah elemen bersama antara himpunan yang diprediksi positif (dalam hal ini, label yang sama dengan pos_label) dan himpunan yang sebenarnya positif (kelas y_test yang sama dengan pos_label) dibagi dengan jumlah elemen total dalam kedua himpunan (yang diprediksi positif dan yang sebenarnya positif). Dalam notasi matematis, skor Jaccard dapat dihitung sebagai:

$$Jaccard\ Score = \frac{TP}{TP + FP + FN}$$

Lalu ada cara kedua dengan menggunakan confusion matrix dapat membantu dalam mengevaluasi kinerja model klasifikasi dan dapat digunakan untuk menghitung berbagai metrik evaluasi seperti sensitivitas, spesifisitas, akurasi, dan sebagainya. Dengan matriks konfusi, kita dapat mengukur sejauh mana model yang telah dibuat dengan membedakan antara hasil yang benar dan hasil yang salah dalam klasifikasi. Kemudian untuk laporan evaluasi dapat digunakan classification_report yang nantinya akan mencetak metrik-metrik seperti presisi, recall, F1-score, dan dukungan (support) untuk setiap kelas dalam masalah klasifikasi. Dan evaluasi terakhir yaitu log loss adalah salah satu metrik evaluasi yang digunakan untuk mengukur sejauh mana model klasifikasi memprediksi probabilitas dengan benar. Semakin rendah nilai log loss, semakin baik model dalam memprediksi probabilitas kelas. Sebaliknya, nilai yang lebih tinggi menunjukkan bahwa model cenderung memiliki kesalahan dalam memprediksi probabilitas.

F1 Score

F1-Score adalah salah satu metrik evaluasi yang umum digunakan dalam klasifikasi, khususnya dalam konteks klasifikasi biner (dua kelas). Metrik ini mengukur keseimbangan antara presisi (precision) dan recall (sensitivity atau true positive rate) dari sebuah model klasifikasi. F1-Score adalah skor yang berkisar antara 0 hingga 1, dan semakin tinggi nilainya, semakin baik performa model.

F1-Score dihitung dengan menggunakan rumus berikut:

$$\text{F1} - \text{Score} = 2 \text{ x } \frac{Precision\ x\ Recall}{Precision+Recall}$$

Precision adalah rasio antara True Positives (TP) dan total prediksi positif (TP + False Positives (FP)). Ini mengukur sejauh mana hasil positif yang diprediksi oleh model adalah benar. Recall adalah rasio antara True Positives (TP) dan total data positif yang sebenarnya (TP + False Negatives (FN)). Ini mengukur sejauh mana model dapat menemukan semua contoh positif yang sebenarnya.

$$\text{Precision} = \frac{TP}{(TP+FP)}$$

$$\text{Recall} = \frac{TP}{(TP+FN)}$$

F1-Score memberikan keseimbangan antara presisi dan recall. Ini berguna ketika Anda ingin mempertimbangkan kedua aspek tersebut secara bersamaan. F1-Score menjadi rendah jika baik presisi maupun recall rendah. F1-Score sangat berguna dalam situasi di mana Anda ingin menghindari kesalahan false positives dan false negatives, dan Anda ingin keseimbangan yang baik antara keduanya. Namun, F1-Score memiliki kekurangan dalam menangani situasi ketidakseimbangan kelas (imbalance class) di mana salah satu kelas memiliki frekuensi yang sangat rendah. Dalam kasus seperti itu, metrik lain seperti area di bawah kurva Precision-Recall (AUC-PR) atau area di bawah kurva Receiver Operating Characteristic (AUC-ROC) mungkin lebih sesuai.